

# Incorrect Protochunk#setStatus call on chunk generate

[MC-263340](#) CHUNK LOADING

|                 |                            |
|-----------------|----------------------------|
| Status          | Resolved / Fixed           |
| Affected        | 1.20                       |
| Fixed in        | 1.20.1 Release Candidate 1 |
| Reported        | 2023-06-08                 |
| Mojira category | Chunk loading              |

## REPRODUCTION TARGET

Use version:

1.20

Affected versions (1):

1.20

Environment:

Minecraft Java Edition 1.20, launched as an unmodified mapped/decompiled development runtime under a Java debugger. Use the integrated server in a disposable single-player world. The corrective ChunkStatusMixin shown in the report, the 1.20.1 Release Candidate 1 fix version, and any other mod that changes chunk generation must not be loaded.

## PRECONDITIONS

---

- Install and select exactly Minecraft Java Edition 1.20, which is the sole affected version listed by the report.
- Have a Java development/debugging environment whose mapped source for `ChunkStatus#generate` matches the 1.20 implementation quoted in the report. Source line breakpoints and Java step-over operations must work before starting the game.
- Do not install or apply the corrective `ChunkStatusMixin` included in the report; it changes the target call and prevents reproduction of the reported implementation.
- Use a new disposable world so that normal spawn-area generation exercises the chunk-status generation pipeline. The report does not provide a required seed, world preset, game mode, coordinates, or server configuration.
- Configure the debugger so it does not automatically resume after reaching the target `ChunkStatus#generate` completion callback.
- Keep the debugger's local-variable/object inspection view available so the Observation Agent can inspect this, the pre-work center `ChunkAccess` referenced as `$$7/$$3x` in the quoted source, the result `Either` referenced as `$$2x`, and the supplied chunk list `$$6` after the trigger.

## STEPS TO REPRODUCE

### 1 **REPORT**

Select an exact Minecraft Java Edition 1.20 mapped development launch configuration. Confirm that it is not 1.20.1 Release Candidate 1 or any later version.

INPUT `Minecraft Java Edition 1.20`

SUCCESS CUE The selected runtime identifies itself as Java Edition 1.20, the first and only affected version listed in the report.

### 2 **REPORT**

Disable all mods, mixins, patches, or transformed classes that modify ChunkStatus, chunk generation, or chunk conversion. In particular, ensure that the corrective ChunkStatusMixin printed in the report is not present.

SUCCESS CUE The runtime will execute the original 1.20 ChunkStatus#generate implementation quoted in the report.

### 3 **REPORT**

In the debugger, open the mapped ChunkStatus class and locate the generate method by matching the method body against the source excerpt in the report. Locate the thenApply completion callback that runs after generationTask.doWork completes.

INPUT `return this.generationTask.doWork(...).thenApply(($$2x) -> {`

SUCCESS CUE The debugger displays the completion callback containing the ProtoChunk type check and status assignment from the report.

### 4 **REPORT**

Within that completion callback, locate the status-assignment statement that operates on the ProtoChunk derived from the pre-work center chunk.

INPUT `$$3x.setStatus(this);`

SUCCESS CUE The exact status-assignment line is selected in the original 1.20 implementation, not in the corrective mixin.

### 5 **INFERRED**

Set a source breakpoint on the `$$3x.setStatus(this)` statement. If the debugger supports conditional breakpoints, condition it on the current ChunkStatus being FULL. A suitable condition for a mapped runtime is `this == ChunkStatus.FULL`; if that symbol cannot be resolved, use `this.toString().equals("full")` or manually resume non-FULL hits.

INPUT `this == ChunkStatus.FULL`

SUCCESS CUE An enabled breakpoint is visible on the status-assignment line and is configured to stop on the FULL-status invocation, or the executor is prepared to filter hits manually.

### 6 **INFERRED**

Optionally set a second breakpoint on the next executable statement after the status assignment, at the profiler-finish check. This provides a stable pause location after stepping over the trigger.

INPUT `if ($$8 != null) {`

SUCCESS CUE The debugger can pause immediately after the target status assignment while the generate callback's local variables remain in scope.

## 7 **INFERRED**

Start Minecraft from the selected development launch configuration in debug mode. Allow startup to continue until the Minecraft main menu is responsive.

**SUCCESS CUE** The Java Edition main menu is displayed and the debugger remains attached to the game process.

## 8 **INFERRED**

Create a disposable single-player world using the normal new-world path. Leave the seed, world preset, and game mode at their defaults because the report specifies none. Select Singleplayer, select Create New World, enter any recognizable temporary world name, and select Create New World again.

**INPUT**

Singleplayer -> Create New World -> World Name: MC-263340 reproduction -> Create New World

**SUCCESS CUE** The integrated server begins preparing the new world and generating its spawn-area chunks.

## 9 **REPORT**

Allow world generation to run until the breakpoint in `ChunkStatus#generate` is reached. If an unconditional breakpoint stops for a status other than `FULL`, inspect the current `ChunkStatus`, resume execution, and repeat until the `FULL` invocation reaches the `$$3x.setStatus(this)` line.

**SUCCESS CUE** Execution is paused before `$$3x.setStatus(this)`, the current `ChunkStatus` is `FULL`, and the branch-local target `$$3x` is a `ProtoChunk` obtained from the center `ChunkAccess` captured before `generationTask.doWork`.

## 10 **REPORT**

Confirm that the debugger is paused inside the `thenApply` callback after `generationTask.doWork` has completed and that the result `Either` represented by `$$2x` is already available in the current stack frame. Do not alter any object fields or evaluate methods with side effects.

**SUCCESS CUE** The callback's pre-work chunk reference and completed result are both available for later inspection, and execution remains paused immediately before the status-assignment call.

## 11 **REPORT** **TRIGGER**

Use the debugger's Step Over operation exactly once to execute the original 1.20 status-assignment statement without entering `ProtoChunk#setStatus`.

**INPUT** Step Over: `$$3x.setStatus(this);`

**SUCCESS CUE** The status-assignment call has just returned, and execution is paused on the following statement in the same `ChunkStatus#generate` completion callback.

## HANDOFF TO OBSERVATION

---

The original Java Edition 1.20 call `$$3x.setStatus(this)` has just executed for the FULL ChunkStatus, and the debugger is paused immediately afterward with the callback frame and its pre-work chunk reference, completed result Either, and chunk list still available. Hand control to the Observation Agent before resuming world generation or saving and closing the world.

## UNCERTAINTIES

---

- The report contains no formal player-facing Steps to Reproduce. This plan therefore uses a debugger to execute and expose the exact internal call identified by the report rather than inventing a gameplay symptom.
- The report does not specify a launcher, mod loader, mappings distribution, decompiler, IDE, debugger, or build configuration. An existing mapped 1.20 development runtime is therefore required, but the exact tooling cannot be selected from the supplied evidence.
- The local names `$$2x`, `$$3x`, `$$6`, `$$7`, and `$$8` come from the decompiled source in the report and may differ in another mapped or decompiled environment. The executor must identify variables by their roles and by matching the quoted method body if those names are unavailable.
- The report does not state whether a fresh world, an existing world, or both reliably leave the misidentified chunk in a saved or externally visible state. A fresh world is selected only as the simplest deterministic way to exercise chunk generation.
- A comment identifies chunk saving as a manifestation, but the report supplies no deterministic save/unload timing that guarantees a particular affected chunk will be written while misidentified. The plan stops at the incorrect internal call rather than fabricating a save-based procedure.
- The report's corrective mixin is evidence about the target implementation but is not itself a reproduction prerequisite; loading it would prevent the original call from being exercised.

MANUAL RESULT

|                  |                    |
|------------------|--------------------|
| Version used     |                    |
| Reproduced?      | YES / NO / PARTIAL |
| Trigger reached? | YES / NO           |
| Crash file       |                    |
| Notes            |                    |
| Tested by / date |                    |